

Prerequisites

Ubuntu26.04 / Ubuntu24.04

For Buildroot::

```
sudo apt-get install git make gcc g++ bzip2 cpio unzip libncurses-dev
```

For QEMU::

```
sudo apt-get install python3-pip python3-venv python3-sphinx python3-tomli libglib2.0-dev ninja-build libpixman-1-dev libslirp-dev
```

For ARM-FastModel::

```
sudo apt-get install xterm
```

```
Fix the lmutil: sudo ln -s /lib64/ld-linux-x86-64.so.2 /lib64/ld-lsb-x86-64.so.3
```

For Terminal::

```
sudo apt-get install gnome-terminal (Default is gnome-shell since ubuntu26)
```

Ubuntu22.04

For Buildroot::

```
sudo apt-get install git make gcc g++ bzip2 cpio unzip libncurses-dev
```

For QEMU::

```
sudo apt-get install python3-pip python3-venv python3-sphinx python3-tomli libglib2.0-dev ninja-build libpixman-1-dev libslirp-dev
```

For ARM-FastModel::

```
sudo apt-get install xterm
```

```
Fix the lmutil: sudo ln -s /lib64/ld-linux-x86-64.so.2 /lib64/ld-lsb-x86-64.so.3
```

For Terminal::

```
sudo apt-get install gnome-terminal
```

Ubuntu20.04

For Buildroot::

```
sudo apt-get install git make gcc g++ bzip2 cpio unzip libncurses-dev
```

For QEMU:: (ubuntu20 must use the **stable-9.0** or older version)

```
sudo apt-get install python3-pip python3-venv python3-sphinx libglib2.0-dev ninja-build libpixman-1-dev libslirp-dev
```

For ARM-FastModel::

```
sudo apt-get install xterm
```

```
Fix the lmutil: sudo ln -s /lib64/ld-linux-x86-64.so.2 /lib64/ld-lsb-x86-64.so.3
```

For Terminal::

```
sudo apt-get install gnome-terminal
```

Quickest & Easiest Way

```
git clone https://github.com/mbedtee/mbedtee-build.git
cd mbedtee-build && ./build.sh aarch64 run
```

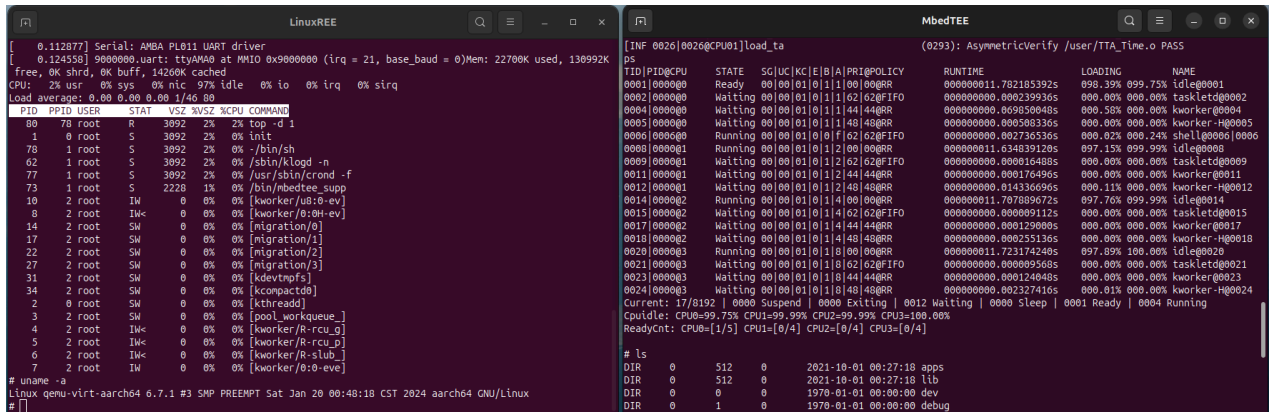
Note

build.sh is just an example reference for the quick-build/run, **please refer to the following chapters for the detailed build/run commands.**

- ./build.sh aarch64 - build for the QEMU virt aarch64 platform (**build only**)
- ./build.sh aarch64 run - build and run the QEMU virt aarch64 platform
- ./build.sh aarch32 run - build and run the QEMU virt aarch32(arm) platform
- ./build.sh riscv64 run - build and run the QEMU virt riscv64 platform
- ./build.sh riscv32 run - build and run the QEMU virt riscv32 platform
- ./build.sh mips32 run - build and run the QEMU malta mips32 platform

Take the QEMU virt AArch64 as example:

- launches 2 gnome-terminal windows, one is for Linux@REE, another one is for the MbedTEE



```
[ 0.112877] Serial: AMBA PL011 UART driver
[ 0.124553] 9600000,uart: ttyAMA0 at MMIO 0x9000000 (irq = 21, base_baud = 0)Mem: 22700K used, 130992K
free: 0K shrd, 0K buff, 14260K cached
CPU: 2% usr 0% sys 0% nic 97% idle 0% io 0% irq 0% irq
Load average: 0.00 0.00 0.00 1/46 80

PID PPID USER STAT VSZ VMSZ %CPU COMMAND
80 78 root R 3092 2K 0% /usr/sbin/cron -f
1 0 root S 3092 2K 0% /bin/sh
78 1 root S 3092 2K 0% /bin/sh
62 1 root S 3092 2K 0% /usr/sbin/cron -f
77 1 root S 3092 2K 0% /usr/sbin/cron -f
73 1 root S 2228 1K 0% /bin/mbedtee supp
10 2 root IW 0 0% 0% [kworker/u8:0-evt]
8 2 root IW 0 0% 0% [kworker/0:0H-evt]
14 2 root SW 0 0% 0% [migration/0]
17 2 root SW 0 0% 0% [migration/1]
22 2 root SW 0 0% 0% [migration/2]
27 2 root SW 0 0% 0% [migration/3]
31 2 root SW 0 0% 0% [kdevtmpfs]
34 2 root SW 0 0% 0% [kcompactd0]
2 0 root SW 0 0% 0% [kthread]
3 2 root SW 0 0% 0% [pool_workqueue_]
4 2 root IW 0 0% 0% [kworker/R-rcu_g]
5 2 root IW 0 0% 0% [kworker/R-rcu_p]
6 2 root IW 0 0% 0% [kworker/R-slub_]
7 2 root IW 0 0% 0% [kworker/0:0-evt]

# uname -a
Linux qemu-virt-aarch64 6.7.1 #3 SMP PREEMPT Sat Jan 20 00:48:18 CST 2024 aarch64 GNU/Linux
# []

[INF 0026@CPU01]load_ta
PS
TID|PID|CPU STATE SG|UC|KC|E|B|A|PRI|POLICY RUNTIME LOADING NAME
0001|000000 Ready 00|00|01|01|1|00|00RR 00000000.7821853925 098.39% 099.75% idleg0001
0002|000000 Waiting 00|00|01|01|1|02|02FIFO 00000000.0002399365 000.00% 000.00% taskletd0002
0004|000000 Waiting 00|00|01|01|1|14|44RR 00000000.0098500405 000.58% 000.00% kworker0004
0005|000000 Waiting 00|00|01|01|1|40|40RR 00000000.0005003365 000.00% 000.00% kworker-H0005
0006|000000 Running 00|00|01|01|0|f02|02FIFO 00000000.0027365365 000.02% 000.24% shell0006|0006
0008|000001 Running 00|00|01|01|2|00|00RR 00000001.6348391205 097.15% 099.99% idleg0008
0009|000001 Waiting 00|00|01|01|2|02|02FIFO 00000000.0000164885 000.00% 000.00% taskletd0009
0011|000001 Waiting 00|00|01|01|2|44|44RR 00000000.0001764965 000.00% 000.00% kworker0011
0012|000001 Waiting 00|00|01|01|2|40|40RR 00000000.0143360965 000.11% 000.00% kworker-H0012
0014|000002 Running 00|00|01|01|4|00|00RR 00000001.7878896725 097.76% 099.99% idleg0014
0015|000002 Waiting 00|00|01|01|4|02|02FIFO 00000000.0000911125 000.00% 000.00% taskletd0015
0017|000002 Waiting 00|00|01|01|4|44|44RR 00000000.0001290005 000.00% 000.00% kworker0017
0018|000002 Waiting 00|00|01|01|4|48|48RR 00000000.0002551365 000.00% 000.00% kworker-H0018
0020|000003 Running 00|00|01|01|0|00|00RR 00000001.7231742405 097.89% 100.00% idleg0020
0021|000003 Waiting 00|00|01|01|0|02|02FIFO 00000000.0000956085 000.00% 000.00% taskletd0021
0023|000003 Waiting 00|00|01|01|0|44|44RR 00000000.0001240485 000.00% 000.00% kworker0023
0024|000003 Waiting 00|00|01|01|0|48|48RR 00000000.0023274165 000.01% 000.00% kworker-H0024

Current: 17/0192 | 0000 Suspend | 0000 Exiting | 0012 Waiting | 0000 Sleep | 0001 Ready | 0004 Running
Cpuidle: CPU0=99.75% CPU1=99.99% CPU2=99.99% CPU3=100.00%
ReadyCnt: CPU0=[1/5] CPU1=[0/4] CPU2=[0/4] CPU3=[0/4]

# ls
DIR 0 512 0 2021-10-01 00:27:18 apps
DIR 0 512 0 2021-10-01 00:27:18 lib
DIR 0 0 0 1970-01-01 00:00:00 dev
DIR 0 1 0 1970-01-01 00:00:00 debug
```

- Following example command is for **run only**, xterm is also an alternative terminal and can be launched with similar options.
- If your system does not have gnome-terminal and xterm, please directly run the qemu-system-xxx command and telnet to the corresponding port to use the REE/TEE terminal.

```
gnome-terminal -e "telnet 127.0.0.1 5555" --tab -t "LinuxREE"& gnome-terminal -e "telnet 127.0.0.1 5556" --tab -t "MbedTEE"&
qemu/build/qemu-system-aarch64 -M virt -M secure=on,gic-version=3,virtualization=on -cpu cortex-a710 -smp 4 -m 4096 -device
loader,file=buildroot/output/images/mbedtee.bin,addr=0x80000000,force-raw=on -device
loader,file=buildroot/output/images/linux.dtb,addr=0x85F00000,force-raw=on -device
loader,file=buildroot/output/images/Image,addr=0x86000000,force-raw=on -device loader,addr=0x80000000,cpu-num=0 -device
loader,addr=0x80000000,cpu-num=1 -device loader,addr=0x80000000,cpu-num=2 -device loader,addr=0x80000000,cpu-num=3 -serial
telnet::5555,server,nowait -serial telnet::5556,server,nowait
```

MbedTEE

Repositories

https://github.com/mbedtee/mbedtee-build.git	-> entry of building the mbedtee
https://github.com/mbedtee/mbedtee-docs.git	-> documentation of mbedtee
https://github.com/mbedtee/mbedtee-os.git	-> kernel of mbedtee
https://github.com/mbedtee/mbedtee-client-api.git	-> GlobalPlatform style client API @ Linux REE
https://github.com/mbedtee/mbedtee-suppl.git	-> REEFS Suppliment System @ Linux UserSpace
https://github.com/mbedtee/mbedtee-crypto.git	-> Cryptographic algorithms for TA encryption and signing
https://github.com/mbedtee/mbedtee-linux-dts.git	-> DTS for Linux REE
https://github.com/mbedtee/mbedtee-globalplatform-client.git	-> GlobalPlatform TestSuite client application @ Linux REE
https://github.com/mbedtee/mbedtee-globalplatform-ta.git	-> GlobalPlatform TestSuite TTAs @ MbedTEE
https://github.com/mbedtee/mbedtee-helloworld-client.git	-> HelloWorld client application @ Linux REE
https://github.com/mbedtee/mbedtee-helloworld-ta.git	-> HelloWorld TA @ MbedTEE

Menuconfig

MbedTEE provides the menuconfig for the feature configuration.

Enter the **buildroot** folder (**mbedtee-build/buildroot**), and issue the command `make mbedtee-os-menuconfig`

```
mbedtee 1.0.1
Architecture Choice (RISCV) --->
RISCV Architecture --->
Platform Configure --->
General Setup --->
IPC --->
Device Drivers --->
File Systems --->
Libraries --->
UserSpace --->
Applications --->

[Space/Enter] Toggle/enter  [ESC] Leave menu          [S] Save
[O] Load                    [?] Symbol info            [/] Jump to symbol
[F] Toggle show-help mode   [C] Toggle show-name mode  [A] Toggle show-all mode
[Q] Quit (prompts for save) [D] Save minimal config (advanced)
```

Build and Run @ QEMU

Prepare the buildroot

```
git clone https://github.com/mbedtee/mbedtee-build.git
cd mbedtee-build && ./buildroot.sh
```

Configure and Build

Enter the **buildroot** folder, @ **mbedtee-build/buildroot**.

Select one of the following configs, issue the command (e.g. for qemu_virt_aarch64):

```
cd buildroot && make mbedtee_qemu_virt_aarch64_defconfig && make
```

Configs to be selected for QEMU platforms:

```
kapa@ubuntu24:~/mbedtee-build/buildroot$ tree configs/ | grep mbedtee_qemu
|-- mbedtee_qemu_ast2600_defconfig
|-- mbedtee_qemu_ast2700_defconfig
|-- mbedtee_qemu_andes_ae350_riscv32_defconfig
|-- mbedtee_qemu_andes_ae350_riscv32_imsic_defconfig
|-- mbedtee_qemu_andes_ae350_riscv32_imsic_linux_defconfig
|-- mbedtee_qemu_andes_ae350_riscv64_defconfig
|-- mbedtee_qemu_andes_ae350_riscv64_imsic_defconfig
|-- mbedtee_qemu_andes_ae350_riscv64_imsic_linux_defconfig
|-- mbedtee_qemu_imx7d_defconfig
|-- mbedtee_qemu_malta_mips32r2_defconfig
|-- mbedtee_qemu_microblaze_v_riscv32_defconfig
|-- mbedtee_qemu_microblaze_v_riscv64_defconfig
|-- mbedtee_qemu_raspi4b_defconfig
|-- mbedtee_qemu_sifive_u_riscv32_defconfig
|-- mbedtee_qemu_sifive_u_riscv64_defconfig
|-- mbedtee_qemu_virt_aarch64_defconfig
|-- mbedtee_qemu_virt_aarch64_emmc_defconfig
|-- mbedtee_qemu_virt_arm_defconfig
|-- mbedtee_qemu_virt_arm_emmc_defconfig
|-- mbedtee_qemu_virt_riscv32_defconfig
|-- mbedtee_qemu_virt_riscv32_imsic_linux_defconfig
|-- mbedtee_qemu_virt_riscv64_defconfig
|-- mbedtee_qemu_virt_riscv64_aplic_defconfig
|-- mbedtee_qemu_virt_riscv64_imsic_linux_defconfig
|-- mbedtee_qemu_virt_riscv64_thread_c906_defconfig
|-- mbedtee_qemu_xiangshan_kmh_linux_defconfig
|-- mbedtee_qemu_xlnx_versal_virt_defconfig
`-- mbedtee_qemu_xlnx_zcu102_defconfig
```

Prepare the QEMU

Enter the **mbedtee-build** folder, clone the QEMU repository to '**mbedtee-build/qemu**'.

```
git clone https://gitlab.com/qemu-project/qemu.git && cd qemu
if [ ! -z `cat /etc/os-release | grep VERSION_ID | grep 20` ]; then git checkout stable-9.0; else git checkout stable-11.0; fi
./configure --prefix=$(pwd)/build --enable-slirp --target-list=mipsel-softmmu,aarch64-softmmu,arm-softmmu,riscv32-softmmu,riscv64-softmmu
make -j8 && make install
```

Warning

ubuntu20 must use the **stable-9.0** or older version, ubuntu22/24/26 can use the **stable-11.0** or newer version.

For **Andes AE350** platforms, clone the Andes QEMU fork to '**mbedtee-build/andes-qemu**':

```
git clone https://github.com/andestech/qemu.git andes-qemu && cd andes-qemu
git checkout ast-v5_4_1-release
./configure --prefix=$(pwd)/build --target-list=riscv32-softmmu,riscv64-softmmu
make -j8 && make install
```

Run the targets @ QEMU

Note

The following configs build/run the targets with MMU/UserSpace enabled, developer can disable MMU/UserSpace to run only the kernel-mode through the [menuconfig](#) (or disable the RISC-V Supervisor-Mode/MMU/UserSpace, run only the RISC-V Machine-Mode).

Important

Switch to the **buildroot** directory (**mbedtee-build/buildroot**) to do the config and make.

"make clean" is required before switching to another target.

AArch64 (REE+TEE)

Config/Make: make clean && make mbedtee_qemu_virt_aarch64_defconfig && make

QEMU virt AArch64 targets: -M virt -cpu cortex-a35/cortex-a53/cortex-a55/cortex-a57/cortex-a72/cortex-a76/cortex-a710/neoverse-n1/neoverse-v1/neoverse-n2

```
gnome-terminal -e "telnet 127.0.0.1 5555" --tab -t "LinuxREE"& gnome-terminal -e "telnet 127.0.0.1 5556" --tab -t "MbedTEE"&
../qemu/build/qemu-system-aarch64 -M virt -M secure=on,gic-version=3,virtualization=on -cpu cortex-a710 -smp 4 -m 4096 -device
loader,file=output/images/mbedtee.bin,addr=0x80000000,force-rw=on -device
loader,file=output/images/linux.dtb,addr=0x85F00000,force-rw=on -device loader,file=output/images/Image,addr=0x86000000,force-
raw=on -device loader,addr=0x80000000,cpu-num=0 -device loader,addr=0x80000000,cpu-num=1 -device loader,addr=0x80000000,cpu-num=2
-device loader,addr=0x80000000,cpu-num=3 -serial telnet::5555,server,nowait -serial telnet::5556,server,nowait
```

or use the xterm terminal: (If your system does not have gnome-terminal and xterm, please directly run the qemu-system-xxx command and telnet to the corresponding port to use the REE/TEE terminal.)

```
xterm -geometry 128x32 -e "telnet localhost 5556" & xterm -geometry 128x32 -e "telnet localhost 5555" & ../qemu/build/qemu-
system-aarch64 -M virt -M secure=on,gic-version=3,virtualization=on -cpu cortex-a710 -smp 4 -m 4096 -device
loader,file=output/images/mbedtee.bin,addr=0x80000000,force-rw=on -device
loader,file=output/images/linux.dtb,addr=0x85F00000,force-rw=on -device loader,file=output/images/Image,addr=0x86000000,force-
raw=on -device loader,addr=0x80000000,cpu-num=0 -device loader,addr=0x80000000,cpu-num=1 -device loader,addr=0x80000000,cpu-num=2
-device loader,addr=0x80000000,cpu-num=3 -serial telnet::5555,server,nowait -serial telnet::5556,server,nowait
```

AArch32 (REE+TEE)

This section covers two common QEMU platforms for ARM AArch32: "-M virt" and "-M vexpress-a15". Additional AArch32 platforms (AST2600, imx7d, etc.) are documented in later sections.

QEMU virt AArch32 targets: -M virt -cpu cortex-a15 or cortex-a7

Config/Make: make clean && make mbedtee_qemu_virt_arm_defconfig && make

```
gnome-terminal -e "telnet 127.0.0.1 5555" --tab -t "LinuxREE"& gnome-terminal -e "telnet 127.0.0.1 5556" --tab -t "MbedTEE"&
../qemu/build/qemu-system-arm -M virt -M secure=on -cpu cortex-a15 -smp 4 -m 2048 -device
loader,file=output/images/mbedtee.bin,addr=0x80000000,force-rw=on -device
loader,file=output/images/linux.dtb,addr=0x85F00000,force-rw=on -device loader,file=output/images/Image,addr=0x86008000,force-
raw=on -device loader,addr=0x80000000,cpu-num=0 -device loader,addr=0x80000000,cpu-num=1 -device loader,addr=0x80000000,cpu-
num=2 -device loader,addr=0x80000000,cpu-num=3 -serial telnet::5555,server,nowait -serial telnet::5556,server,nowait
```

QEMU vexpress AArch32 target: -M vexpress-a15 -cpu cortex-a15

Config/Make: make clean && make mbedtee_vexpress_ca15_defconfig && make

```
xterm -geometry 128x32 -e "telnet 127.0.0.1 5555" & xterm -geometry 128x32 -e "telnet 127.0.0.1 5556" & xterm -geometry 128x32 -
e "telnet 127.0.0.1 5557" & ../qemu/build/qemu-system-arm -M vexpress-a15 -cpu cortex-a15 -smp 4 -m 2G -device
loader,file=output/images/mbedtee.bin,addr=0x80000000,force-rw=on -device
loader,file=output/images/linux.dtb,addr=0x85F00000,force-rw=on -device loader,file=output/images/Image,addr=0x86008000,force-
raw=on -device loader,addr=0x80000000,cpu-num=0 -device loader,addr=0x80000000,cpu-num=1 -device loader,addr=0x80000000,cpu-num=2
-device loader,addr=0x80000000,cpu-num=3 -serial telnet::5555,server,nowait -serial telnet::5556,server,nowait -serial
telnet::5557,server,nowait
```

AArch64 (REE+TEE with eMMC/RPMB)

Config/Make: make clean && make mbedtee_qemu_virt_aarch64_emmc_defconfig && make

QEMU virt AArch64 targets: -M virt -cpu cortex-a35/cortex-a53/cortex-a55/cortex-a57/cortex-a72/cortex-a76/cortex-a710/neoverse-n1/neoverse-v1/neoverse-n2

Run with eMMC enabled @ REE side, RPMB enabled @ TEE side.

Note

Current QEMU does not support RPMB batch write, so please "make mbedtee-os-menuconfig" to set the batch-write-number to 1.

```
xterm -geometry 128x32 -e "telnet localhost 5556" & xterm -geometry 128x32 -e "telnet localhost 5555"& ../qemu/build/qemu-system-aarch64 -M virt -M secure=on,gic-version=3,virtualization=on -cpu cortex-a710 -smp 4 -m 4096 -device loader,file=output/images/mbedtee.bin,addr=0x80000000,force-raw=on -device loader,file=output/images/linux.dtb,addr=0x85F00000,force-raw=on -device loader,file=output/images/Image,addr=0x86000000,force-raw=on -device loader,addr=0x80000000,cpu-num=0 -device loader,addr=0x80000000,cpu-num=1 -device loader,addr=0x80000000,cpu-num=2 -device loader,addr=0x80000000,cpu-num=3 -drive file=output/images/emmc.img,if=none,format=raw,id=emmc-img -device sdhci-pci -device emmc,boot-partition-size=1048576,rpmb-partition-size=1048576,drive=emmc-img -serial telnet::5555,server,nowait -serial telnet::5556,server,nowait
```

AArch32 (REE+TEE with eMMC/RPMB)

Config/Make: make clean && make mbedtee_qemu_virt_arm_emmc_defconfig && make

QEMU virt AArch32 targets: -M virt -cpu cortex-a15 or cortex-a7

Run with eMMC enabled @ REE side, RPMB enabled @ TEE side.

Note

Current QEMU does not support RPMB batch write, so please "make mbedtee-os-menuconfig" to set the batch-write-number to 1.

```
xterm -geometry 128x32 -e "telnet localhost 5555" & xterm -geometry 128x32 -e "telnet localhost 5556"& ../qemu/build/qemu-system-arm -M virt -M secure=on -cpu cortex-a15 -smp 4 -m 2048 -M highmem=on -device loader,file=output/images/mbedtee.bin,addr=0x80000000,force-raw=on -device loader,file=output/images/linux.dtb,addr=0x85F00000,force-raw=on -device loader,file=output/images/Image,addr=0x86008000,force-raw=on -device loader,addr=0x80000000,cpu-num=0 -device loader,addr=0x80000000,cpu-num=1 -device loader,addr=0x80000000,cpu-num=2 -device loader,addr=0x80000000,cpu-num=3 -drive file=output/images/emmc.img,if=none,format=raw,id=emmc-img -device sdhci-pci -device emmc,boot-partition-size=1048576,rpmb-partition-size=1048576,drive=emmc-img -serial telnet::5555,server,nowait -serial telnet::5556,server,nowait
```

RISCV64 (TEE only)

Config/Make: make clean && make mbedtee_qemu_virt_riscv64_defconfig && make

```
../qemu/build/qemu-system-riscv64 -M virt -smp 4 -m 8G -device loader,file=output/images/mbedtee.bin,addr=0x80000000,force-raw=on -device loader,addr=0x80000000,cpu-num=0 -device loader,addr=0x80000000,cpu-num=1 -device loader,addr=0x80000000,cpu-num=2 -device loader,addr=0x80000000,cpu-num=3 -M aclint=on -serial stdio
```

RISCV64-APLIC (TEE only)

Run with APLIC direct mode interrupt controller.

Config/Make: make clean && make mbedtee_qemu_virt_riscv64_aplic_defconfig && make

```
../qemu/build/qemu-system-riscv64 -M virt,aia=aplic,aclint=on -smp 4 -m 4G -device loader,file=output/images/mbedtee.bin,addr=0x80000000,force-raw=on -device loader,addr=0x80000000,cpu-num=0 -device loader,addr=0x80000000,cpu-num=1 -device loader,addr=0x80000000,cpu-num=2 -device loader,addr=0x80000000,cpu-num=3 -serial stdio
```

RISCV64 T-Head C906 (TEE only)

Run with T-Head XuanTie C906 single-core CPU (RV64GCV, xtheadcmo cache ops, PLIC).

Config/Make: make clean && make mbedtee_qemu_virt_riscv64_thead_c906_defconfig && make

```
../qemu/build/qemu-system-riscv64 -M virt -cpu thead-c906 -smp 1 -m 4G -device  
loader,file=output/images/mbedtee.bin,addr=0x80000000,force-raw=on -device loader,addr=0x80000000,cpu-num=0 -M aclint=on -serial  
stdio
```

RISCV32 (TEE only)

Config/Make: make clean && make mbedtee_qemu_virt_riscv32_defconfig && make

```
../qemu/build/qemu-system-riscv32 -M virt,aclint=on -smp 4 -m 2G -device  
loader,file=output/images/mbedtee.bin,addr=0x80000000,force-raw=on -device loader,addr=0x80000000,cpu-num=0 -device  
loader,addr=0x80000000,cpu-num=1 -device loader,addr=0x80000000,cpu-num=2 -device loader,addr=0x80000000,cpu-num=3 -serial stdio
```

SifiveUnleashed RISCV32 (TEE only)

Config/Make: make clean && make mbedtee_qemu_sifive_u_riscv32_defconfig && make

```
../qemu/build/qemu-system-riscv32 -M sifive_u -smp 5 -m 2G -device loader,file=output/images/mbedtee.bin,addr=0x80000000,force-  
raw=on -device loader,addr=0x80000000,cpu-num=0 -device loader,addr=0x80000000,cpu-num=1 -device loader,addr=0x80000000,cpu-num=2  
-device loader,addr=0x80000000,cpu-num=3 -device loader,addr=0x80000000,cpu-num=4 -serial stdio
```

SifiveUnleashed RISCV64 (TEE only)

Config/Make: make clean && make mbedtee_qemu_sifive_u_riscv64_defconfig && make

```
../qemu/build/qemu-system-riscv64 -M sifive_u -smp 5 -m 4G -device loader,file=output/images/mbedtee.bin,addr=0x80000000,force-  
raw=on -device loader,addr=0x80000000,cpu-num=0 -device loader,addr=0x80000000,cpu-num=1 -device loader,addr=0x80000000,cpu-num=2  
-device loader,addr=0x80000000,cpu-num=3 -device loader,addr=0x80000000,cpu-num=4 -serial stdio
```

Andes AE350 RISCV32 (TEE only)

Supported CPUs: andes-a25, andes-a27, andes-a45, andes-a46mp, andes-a46mpv

Config/Make: make clean && make mbedtee_qemu_andes_ae350_riscv32_defconfig && make

```
../andes-qemu/build/qemu-system-riscv32 -M andes_ae350 -cpu andes-a45 -smp 4 -m 2G -bios none -device  
loader,file=output/images/mbedtee.bin,addr=0x0,force-raw=on -serial null -serial stdio
```

Andes AE350 RISCV32-IMSIC (TEE only)

Run with APLIC-IMSIC interrupt controller.

Config/Make: make clean && make mbedtee_qemu_andes_ae350_riscv32_imsic_defconfig && make

```
../andes-qemu/build/qemu-system-riscv32 -M andes_ae350,aia=aplic-imsic -cpu andes-a45 -smp 4 -m 2G -bios none -device  
loader,file=output/images/mbedtee.bin,addr=0x0,force-raw=on -serial null -serial stdio
```

Andes AE350 RISCv64 (TEE only)

Supported CPUs: andes-ax25, andes-ax27, andes-ax45, andes-ax45mpv, andes-ax46mp, andes-ax46mpv, andes-ax65, andes-ax66

Config/Make: make clean && make mbedtee_qemu_andes_ae350_riscv64_defconfig && make

```
../andes-qemu/build/qemu-system-riscv64 -M andes_ae350 -cpu andes-ax45 -smp 4 -m 2G -bios none -device loader,file=output/images/mbedtee.bin,addr=0x0,force-raw=on -serial null -serial stdio
```

Andes AE350 RISCv64-IMSIc (TEE only)

Run with APLIC-IMSIc interrupt controller.

Config/Make: make clean && make mbedtee_qemu_andes_ae350_riscv64_imsic_defconfig && make

```
../andes-qemu/build/qemu-system-riscv64 -M andes_ae350,aia=aplic-imsic -cpu andes-ax45 -smp 4 -m 2G -bios none -device loader,file=output/images/mbedtee.bin,addr=0x0,force-raw=on -serial null -serial stdio
```

Andes AE350 RISCv32-IMSIc (REE+TEE)

Run with '-smp 4', 2 cores for LinuxREE, 2 cores for MbedTEE. APLIC MSI mode.

Config/Make: make clean && make mbedtee_qemu_andes_ae350_riscv32_imsic_linux_defconfig && make

```
../andes-qemu/build/qemu-system-riscv32 -M andes_ae350,aia=aplic-imsic -cpu andes-a45 -smp 4 -m 2G -bios none -device loader,file=output/images/fw_jump.bin,addr=0x6000000,force-raw=on -device loader,file=output/images/Image,addr=0x6400000,force-raw=on -device loader,file=output/images/mbedtee.bin,addr=0x400000,force-raw=on -device loader,addr=0x400000,cpu-num=0 -device loader,addr=0x400000,cpu-num=1 -device loader,addr=0x6000000,cpu-num=2 -device loader,addr=0x6000000,cpu-num=3 -serial null -serial stdio
```

Andes AE350 RISCv64-IMSIc (REE+TEE)

Run with '-smp 8', 4 cores for LinuxREE, 4 cores for MbedTEE. APLIC MSI mode.

Config/Make: make clean && make mbedtee_qemu_andes_ae350_riscv64_imsic_linux_defconfig && make

```
../andes-qemu/build/qemu-system-riscv64 -M andes_ae350,aia=aplic-imsic -cpu andes-ax45 -smp 8 -m 2G -bios none -device loader,file=output/images/fw_jump.bin,addr=0x6000000,force-raw=on -device loader,file=output/images/Image,addr=0x6200000,force-raw=on -device loader,file=output/images/mbedtee.bin,addr=0x200000,force-raw=on -device loader,addr=0x200000,cpu-num=0 -device loader,addr=0x200000,cpu-num=1 -device loader,addr=0x200000,cpu-num=2 -device loader,addr=0x200000,cpu-num=3 -device loader,addr=0x6000000,cpu-num=4 -device loader,addr=0x6000000,cpu-num=5 -device loader,addr=0x6000000,cpu-num=6 -device loader,addr=0x6000000,cpu-num=7 -serial null -serial stdio
```

Malta MIPS32 (TEE only)

QEMU malta mips32r2 targets: -cpu 24Kf 24Kc 34Kf 74Kf M14Kc P5600

Config/Make: make clean && make mbedtee_qemu_malta_mips32r2_defconfig && make

```
../qemu/build/qemu-system-mipsel -M malta -m 1G -cpu 74Kf -kernel output/images/mbedtee.elf -serial stdio
```

ASpeed AST2600 (REE+TEE)

Config/Make: make clean && make mbedtee_qemu_ast2600_defconfig && make

```
../qemu/build/qemu-system-arm -M ast2600-evb -smp 2 -m 2G -device loader,file=output/images/mbedtee.bin,addr=0x80000000,force-raw=on -device loader,file=output/images/linux.dtb,addr=0x85F00000,force-raw=on -device loader,file=output/images/Image,addr=0x86008000,force-raw=on -device loader,addr=0x80000000,cpu-num=0 -device loader,addr=0x80000000,cpu-num=1 -serial stdio
```


ASpeed AST2700 (REE+TEE)

Config/Make: make clean && make mbedtee_qemu_ast2700_defconfig && make CONFIG_ARCH_ASPEED=y

⚠ Warning

`CONFIG_ARCH_ASPEED=y` **MUST be passed on the make command line -- not just in .config.**

Forgetting it will produce a build that does NOT support the AST2700 SoC specifics (e.g. GIC, UART, memory map).

The correct full sequence is:

```
make clean && make mbedtee_qemu_ast2700_defconfig && make CONFIG_ARCH_ASPEED=y
```

📌 Note

Add the following GIC patch to the QEMU aspeed_ast2700.c aspeed_soc_ast2700_gic_realize() function and rebuild the QEMU.

```
qdev_prop_set_bit(gicdev, "has-security-extensions", true); /* line 559 */
```

```
../qemu/build/qemu-system-aarch64 -M ast2700-evb -smp 4 -m 2G -device loader,file=output/images/mbedtee.bin,addr=0x400000000,force-raw=on -device loader,file=output/images/linux.dtb,addr=0x415F00000,force-raw=on -device loader,file=output/images/Image,addr=0x420000000,force-raw=on -device loader,addr=0x400000000,cpu-num=0 -device loader,addr=0x400000000,cpu-num=1 -device loader,addr=0x400000000,cpu-num=2 -device loader,addr=0x400000000,cpu-num=3 -serial stdio
```

Raspberry raspi4b (REE+TEE)

Config/Make: make clean && make mbedtee_qemu_raspi4b_defconfig && make

📌 Note

Security extension is necessary when the processor runs both the REE and TEE mode firmwares, so please add the following GIC patch to the QEMU hw/arm/bcm2838.c bcm2838_realize() function and rebuild the QEMU, otherwise the REE will not be launched.

```
object_property_set_bool(OBJECT(&s->gic), "has-security-extensions", true, errp); /* line 127 */
```

```
gnome-terminal -e "telnet 127.0.0.1 5555" --tab -t "LinuxREE" & gnome-terminal -e "telnet 127.0.0.1 5556" --tab -t "MbedTEE" &
../qemu/build/qemu-system-aarch64 -M raspi4b -smp 4 -m 2G -device loader,file=output/images/mbedtee.bin,addr=0x00200000,force-raw=on -device loader,file=output/images/linux.dtb,addr=0x05F00000,force-raw=on -device loader,file=output/images/Image,addr=0x06000000,force-raw=on -device loader,addr=0x00200000,cpu-num=0 -device loader,addr=0x00200000,cpu-num=1 -device loader,addr=0x00200000,cpu-num=2 -device loader,addr=0x00200000,cpu-num=3 -serial telnet::5555,server,nowait -serial telnet::5556,server,nowait
```

Xilinx xlnx-versal-virt (REE+TEE)

Config/Make: make clean && make mbedtee_qemu_xlnx_versal_virt_defconfig && make

```
gnome-terminal -e "telnet 127.0.0.1 5555" --tab -t "LinuxREE" & gnome-terminal -e "telnet 127.0.0.1 5556" --tab -t "MbedTEE" &
../qemu/build/qemu-system-aarch64 -M xlnx-versal-virt -m 2G -device loader,file=output/images/mbedtee.bin,addr=0x00200000,force-raw=on -device loader,file=output/images/linux.dtb,addr=0x05F00000,force-raw=on -device loader,file=output/images/Image,addr=0x06000000,force-raw=on -device loader,addr=0x00200000,cpu-num=0 -device loader,addr=0x00200000,cpu-num=1 -serial telnet::5555,server,nowait -serial telnet::5556,server,nowait
```

Xilinx xlnx-zcu102 (REE+TEE)

Config/Make: make clean && make mbedtee_qemu_xlnx_zcu102_defconfig && make

```
gnome-terminal -e "telnet 127.0.0.1 5555" --tab -t "LinuxREE"& gnome-terminal -e "telnet 127.0.0.1 5556" --tab -t "MbedTEE"&
../qemu/build/qemu-system-aarch64 -M xlnx-zcu102 -M secure=on,virtualization=on -m 2G -device
loader,file=output/images/mbedtee.bin,addr=0x00200000,force-raw=on -device
loader,file=output/images/linux.dtb,addr=0x05F00000,force-raw=on -device loader,file=output/images/Image,addr=0x06000000,force-
raw=on -device loader,addr=0x00200000,cpu-num=0 -device loader,addr=0x00200000,cpu-num=1 -device loader,addr=0x00200000,cpu-num=2
-device loader,addr=0x00200000,cpu-num=3 -serial telnet::5555,server,nowait -serial telnet::5556,server,nowait
```

NXP imx7d (REE+TEE)

Config/Make: make clean && make mbedtee_qemu_imx7d_defconfig && make

```
gnome-terminal -e "telnet 127.0.0.1 5555" --tab -t "LinuxREE"& gnome-terminal -e "telnet 127.0.0.1 5556" --tab -t "MbedTEE"&
../qemu/build/qemu-system-arm -M mcimx7d-sabre -smp 2 -m 2G -device loader,file=output/images/mbedtee.bin,addr=0x80000000,force-
raw=on -device loader,file=output/images/linux.dtb,addr=0x85F00000,force-raw=on -device
loader,file=output/images/Image,addr=0x86008000,force-raw=on -device loader,addr=0x80000000,cpu-num=0 -device
loader,addr=0x80000000,cpu-num=1 -serial telnet::5555,server,nowait -serial telnet::5556,server,nowait
```

RISCV64-IMSI (REE+TEE)

Run with '-smp 8', 4 cores for LinuxREE, 4 cores for MbedTEE. APLIC MSI mode.

Config/Make: make clean && make mbedtee_qemu_virt_riscv64_imsic_linux_defconfig && make

```
../qemu/build/qemu-system-riscv64 -M virt -smp 8 -m 8G -device loader,file=output/images/fw_jump.bin,addr=0x86000000,force-raw=on
-device loader,file=output/images/Image,addr=0x86200000,force-raw=on -device
loader,file=output/images/mbedtee.bin,addr=0x80000000,force-raw=on -device loader,addr=0x80000000,cpu-num=0 -device
loader,addr=0x80000000,cpu-num=1 -device loader,addr=0x80000000,cpu-num=2 -device loader,addr=0x80000000,cpu-num=3 -device
loader,addr=0x86000000,cpu-num=4 -device loader,addr=0x86000000,cpu-num=5 -device loader,addr=0x86000000,cpu-num=6 -device
loader,addr=0x86000000,cpu-num=7 -M aia=aplic-imsic -serial stdio
```

RISCV32-IMSI (REE+TEE)

Run with '-smp 4', 2 cores for LinuxREE, 2 cores for MbedTEE. APLIC MSI mode.

Config/Make: make clean && make mbedtee_qemu_virt_riscv32_imsic_linux_defconfig && make

```
../qemu/build/qemu-system-riscv32 -M virt -smp 4 -m 2G -device loader,file=output/images/fw_jump.bin,addr=0x86000000,force-raw=on
-device loader,file=output/images/Image,addr=0x86400000,force-raw=on -device
loader,file=output/images/mbedtee.bin,addr=0x80000000,force-raw=on -device loader,addr=0x80000000,cpu-num=0 -device
loader,addr=0x80000000,cpu-num=1 -device loader,addr=0x86000000,cpu-num=2 -device loader,addr=0x86000000,cpu-num=3 -M aia=aplic-
imsic -nographic
```

Xiangshan-KMH (REE+TEE)

Run with '-smp 16' RISCV64, 12 cores for LinuxREE, 4 cores for MbedTEE. APLIC MSI mode.

Config/Make: make clean && make mbedtee_qemu_xiangshan_kmh_linux_defconfig && make

```
../qemu/build/qemu-system-riscv64 -M xiangshan-kunminghu -smp 16 -m 8G -device
loader,file=output/images/fw_jump.bin,addr=0x86000000,force-raw=on -device loader,file=output/images/Image,addr=0x86200000,force-
raw=on -device loader,file=output/images/mbedtee.bin,addr=0x80000000,force-raw=on -device loader,addr=0x80000000,cpu-num=0 -
device loader,addr=0x80000000,cpu-num=1 -device loader,addr=0x80000000,cpu-num=2 -device loader,addr=0x80000000,cpu-num=3 -device
loader,addr=0x86000000,cpu-num=4 -device loader,addr=0x86000000,cpu-num=5 -device loader,addr=0x86000000,cpu-num=6 -device
loader,addr=0x86000000,cpu-num=7 -device loader,addr=0x86000000,cpu-num=8 -device loader,addr=0x86000000,cpu-num=9 -device
loader,addr=0x86000000,cpu-num=10 -device loader,addr=0x86000000,cpu-num=11 -device loader,addr=0x86000000,cpu-num=12 -device
loader,addr=0x86000000,cpu-num=13 -device loader,addr=0x86000000,cpu-num=14 -device loader,addr=0x86000000,cpu-num=15 -serial
stdio
```

Microblaze-V (TEE only)

Run with one RISCv64 or RISCv32 imfac core.

RISCv64: make clean && make mbedtee_qemu_microblaze_v_riscv64_defconfig && make

```
../qemu/build/qemu-system-riscv64 -M amd-microblaze-v-generic -device  
loader,file=output/images/mbedtee.bin,addr=0x80000000,force-raw=on -device loader,addr=0x80000000,cpu-num=0 -serial none -serial  
stdio
```

RISCv32: make clean && make mbedtee_qemu_microblaze_v_riscv32_defconfig && make

```
../qemu/build/qemu-system-riscv32 -M amd-microblaze-v-generic -device  
loader,file=output/images/mbedtee.bin,addr=0x80000000,force-raw=on -device loader,addr=0x80000000,cpu-num=0 -serial none -serial  
stdio
```

Note

If the user needs to run the microblaze-v with MMU (Supervisor/User modes), please select the following mbedtee-os defconfig:

qemu_microblaze_v_riscv64_S_defconfig or qemu_microblaze_v_riscv32_S_defconfig

Modify the microblaze-v-generic.c line-88 in QEMU source tree and rebuild the QEMU. Because the Supervisor EIE is 9 instead of 11.

```
sysbus_connect_irq(SYS_BUS_DEVICE(dev), 0, qdev_get_gpio_in(DEVICE(cpu), /* 11 */ 9));
```

Build and Run @ FastModel

Prepare the buildroot

```
git clone https://github.com/mbedtee/mbedtee-build.git
cd mbedtee-build && ./buildroot.sh
```

Configure and Build

Enter the **buildroot** folder, @ **mbedtee-build/buildroot**.

Select one of the following configs, issue the command (e.g. for Cortex-A78x4):

```
cd buildroot && make clean && make mbedtee_vexpress_ca78_defconfig && make
```

Configs to be selected for ARM FastModels:

```
kapa@ubuntu24:~/mbedtee-build/buildroot$ tree configs/ | grep mbedtee_vexpress
|-- mbedtee_vexpress_ca15_defconfig
|-- mbedtee_vexpress_ca17x4_ca7x4_defconfig
|-- mbedtee_vexpress_ca510x4_ca710x4_defconfig
|-- mbedtee_vexpress_ca53_defconfig
|-- mbedtee_vexpress_ca5_defconfig
|-- mbedtee_vexpress_ca65_defconfig
|-- mbedtee_vexpress_ca73x4_ca53x4_defconfig
`-- mbedtee_vexpress_ca78_defconfig
```

Prepare the FastModel

1. Before setting up the LicenseManager and FastModel SystemCanvas, please get a Fast Model Library license from support@arm.com, then generate your **license.dat** from below website.
<https://developer.arm.com/support/licensing/generate>
2. Download the License Management tool - BX002-PT-00007-r11p17-02rel0.tar.gz.
<https://developer.arm.com/Tools%20and%20Software/License%20Management#Downloads>
3. Download "Fast Models 11.23 for Linux x86" from the below ARM website:
<https://developer.arm.com/downloads/view/FM000A?sortBy=availableBy&revision=r11p23-09rel0>
4. Setup and stop/start the license manager (assume your license.dat is in ~/ folder):

```
tar xvf BX002-PT-00007-r11p17-02rel0.tar.gz
./BX002-PT-00007-r11p17-02rel0/lmutil lmdown -c ~/license.dat
./BX002-PT-00007-r11p17-02rel0/lmgrd -c ~/license.dat -l
```

Make sure the Ubuntu MAC and the name of the account currently logged on are consistent with the MAC/Name used for generating license.dat.

When successful, you will see the log: **(armlmd) DPLT: waiting for logger to connect**

5. Setup the "Fast Models 11.23 for Linux x86"

```
tar xvf FastModels_11-23-009_Linux64.tgz
./FastModels_11-23-009_Linux64/setup.bin
```

Specify the installation directory and the location of the license.dat, then click Next until finished.




Please select the base directory for the installation of **Fast Models**

Sub-directories will be created for each software module of the installation package.

Please select the location of the license file:

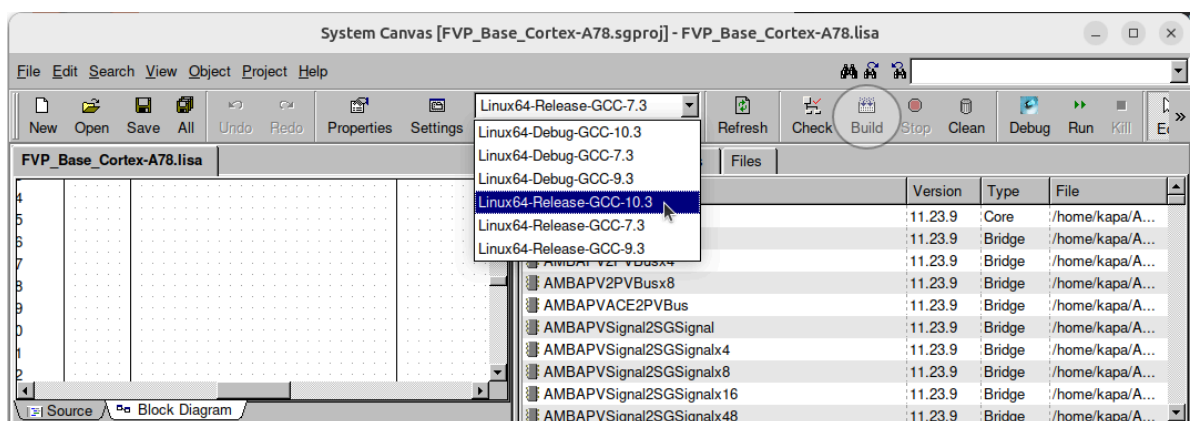
Note: if location is not specified here, the environment variable `ARMLMD_LICENSE_FILE` must be set later

6. Run the FastModel SystemCanvas (Setting the environment path and run the **sgcanvas**)

```
./home/kapa/ARM/FastModelsTools_11.23/source_all.sh
sgcanvas
```

7. Build the FastModel targets (e.g. for AArch64 Cortex-A78)

- Load project: Click the System Canvas "**File**" -> "**Load Project**" -> Select the **ARM/FastModelsPortfolio_11.23/examples/LISA/FVP_Base/Build_Cortex-A78/FVP_Base_Cortex-A78.sgproj**
- Select Active Project Configuration -> **Linux-Release-GCC-10.3**
- Click the **Build**, waiting for 3 ~ 5 minutes for building the target model libraries, check the Log window for success or failure information, "Model Build process completed successfully." means for a successful building.



- Other targets can be built in the similar way.

Run the targets @ FastModel

Important

Switch to the **buildroot** directory (**mbedtee-build/buildroot**) to do the config and make.

"make clean" is required before switching to another target.

AArch64 (CA78 Series)

- Config/Make: make clean && make mbedtee_vexpress_ca78_defconfig && make
- Support targets:

```
FVP_Base_Cortex_A55x4, FVP_Base_Cortex_A75x4, FVP_Base_Cortex_A76x4, FVP_Base_Cortex_A77x4
FVP_Base_Cortex_A78x4, FVP_Base_Cortex_X1x4, FVP_Base_Cortex_A510x4, FVP_Base_Cortex_A710x4
FVP_Base_Cortex_X2x4, FVP_Base_Neoverse_N1x4, FVP_Base_Neoverse_V1x4, FVP_Base_Neoverse_N2x4
```

- System Canvas - Load project: Click the "**File**" -> "**Load Project**" -> Select the **ARM/FastModelsPortfolio_11.23/examples/LISA/FVP_Base/Build_Cortex-A78/FVP_Base_Cortex-A78.sgproj**

- System Canvas - Select Active Project Configuration -> **Linux-Release-GCC-10.3** and then click the **Build** button
- System Canvas - Click the **Run** button -> Select the **ISIM system** -> Input below command to run the target -> Click **OK**

```
"%ISIM%" --data /home/kapa/mbedtee-build/buildroot/output/images/mbedtee.bin@0x80000000 --data /home/kapa/mbedtee-build/buildroot/output/images/linux.dtb@0x85f00000 --data /home/kapa/mbedtee-build/buildroot/output/images/Image@0x86000000 --start FVP_Base_Cortex_A78.cluster0.cpu0=0x80000000 --start FVP_Base_Cortex_A78.cluster0.cpu1=0x80000000 --start FVP_Base_Cortex_A78.cluster0.cpu2=0x80000000 --start FVP_Base_Cortex_A78.cluster0.cpu3=0x80000000 --param bp.secure_memory=false -S --run -c pct1.startup=0.0.*.0 -c bp.vis.rate_limit-enable=false
```

- Command window of Run:

Run

Select command to run

☐ Model Debugger ☐ Model Shell ☒ ISIM system ☐ Custom

Recent: `"%ISIM%" --data /home/kapa/mbedtee-build/buildroot/output/images/mbedtee.bin@0x80000000 --data /home/kapa/mbedtee-build/buildroot/output/images/linux.dtb@0x85f00000 --data /home/kapa/mbedtee-build/buildroot/output/images/Image@0x86000000 --start FVP_Base_Cortex_A78.cluster0.cpu0=0x80000000 --start FVP_Base_Cortex_A78.cluster0.cpu1=0x80000000 --start FVP_Base_Cortex_A78.cluster0.cpu2=0x80000000 --start FVP_Base_Cortex_A78.cluster0.cpu3=0x80000000 --param bp.secure_memory=false -S --run -c pct1.startup=0.0.*.0 -c bp.vis.rate_limit-enable=false`

Full command line

`"%ISIM%" --data /home/kapa/mbedtee-build/buildroot/output/images/mbedtee.bin@0x80000000 --data /home/kapa/mbedtee-build/buildroot/output/images/linux.dtb@0x85f00000 --data /home/kapa/mbedtee-build/buildroot/output/images/Image@0x86000000 --start FVP_Base_Cortex_A78.cluster0.cpu0=0x80000000 --start FVP_Base_Cortex_A78.cluster0.cpu1=0x80000000 --start FVP_Base_Cortex_A78.cluster0.cpu2=0x80000000 --start FVP_Base_Cortex_A78.cluster0.cpu3=0x80000000 --param bp.secure_memory=false -S --run -c`

Insert Placeholder Macro

☐ Launch in background (run asynchronously) ☐ Keep console

Effective command line with expanded macros and environment variables

`"/home/kapa/ARM/FastModelsPortfolio_11.23/examples/LISA/FVP_Base/Build_Cortex-A78/Linux64-Release-GCC-10.3 --data /home/kapa/mbedtee-build/buildroot/output/images/mbedtee.bin@0x80000000 --data /home/kapa/mbedtee-build/buildroot/output/images/linux.dtb@0x85f00000 --data /home/kapa/mbedtee-build/buildroot/output/images/Image@0x86000000 --start FVP_Base_Cortex_A78.cluster0.cpu0=0x80000000 --start FVP_Base_Cortex_A78.cluster0.cpu1=0x80000000 --start FVP_Base_Cortex_A78.cluster0.cpu2=0x80000000 --start FVP_Base_Cortex_A78.cluster0.cpu3=0x80000000 --param bp.secure_memory=false -S --run -c`

Status:

Clear History

OK Cancel

- xterm will be launched for LinuxREE (FVP terminal_0) and MbedTEE (FVP terminal_1):

System Canvas [FVP_Base_Cortex-A78.sproj] - FVP_Base_Cortex-A78.lisa

File Edit Search View Object Project Help

New Open Save All Undo Redo Properties Settings Select Active Project Configuration Refresh Check Build Stop Clean Debug Run Kill Edit Connect Pan In Out

Fast Models - CLCD Cortex-A78 Base FVP

FVP terminal_0

```

# ps
PID TID CPU STATE SCUICWICIEIBIAPRIPOLICY RUNTIME LOADING NAME
0001 00000000 Ready 001001010111100100RR 000000000,400184825s 099.99% 099.99% idle#0001
0002 00000000 Waiting 0010010101111621E22F1FD 000000000,000001036s 000.00% 000.00% tasklet#0002
0004 00000000 Waiting 001001010111144144RR 000000000,007381612s 000.00% 000.00% kworker#0004
0005 00000000 Waiting 001001010111148148RR 000000000,000124382s 000.00% 000.00% kworker#H0005
0006 00000000 Running 0010010101111621E22F1FD 000000000,000523225s 000.00% 000.00% shell#0006
0008 00000001 Running 001001010111210010RR 000000268,133903237s 099.91% 100.00% idle#0008
0009 00000001 Waiting 00100101011121621E22F1FD 000000000,00000589s 000.00% 000.00% tasklet#0009
0011 00000001 Waiting 001001010111214414RR 000000000,000206619s 000.00% 000.00% kworker#0011
0012 00000001 Waiting 001001010111214814RR 000000000,000230701s 000.00% 000.00% kworker#H0012
0014 00000002 Running 001001010111410010RR 000000269,224153136s 099.92% 100.00% idle#0014
0015 00000002 Waiting 00100101011141621E22F1FD 000000000,00000589s 000.00% 000.00% tasklet#0015
0017 00000002 Waiting 001001010111414414RR 000000000,000207858s 000.00% 000.00% kworker#0017
0018 00000002 Waiting 001001010111414814RR 000000000,000119860s 000.00% 000.00% kworker#H0018
0020 00000003 Running 001001010111810010RR 000000269,134763774s 099.91% 099.99% idle#0020
0021 00000003 Waiting 001001010111621E22F1FD 000000000,00000589s 000.00% 000.00% tasklet#0021
0023 00000003 Waiting 001001010111814414RR 000000000,000206010s 000.00% 000.00% kworker#0023
0024 00000003 Waiting 001001010111814814RR 000000000,000120309s 000.00% 000.00% kworker#H0024
Current: 17/8192 I 0000 Suspend I 0000 Exiting I 0012 Waiting I 0000 Sleep I 0001 Ready I 0004 Running
CpuIdle: CPU0=99.99% CPU1=100.00% CPU2=100.00% CPU3=99.99%
ReadyCnt: CPU0=[1/5] CPU1=[0/4] CPU2=[0/4] CPU3=[0/4]

```

FVP terminal_1

```

# ps
PID TID CPU STATE SCUICWICIEIBIAPRIPOLICY RUNTIME LOADING NAME
0001 00000000 Ready 001001010111100100RR 000000000,400184825s 099.99% 099.99% idle#0001
0002 00000000 Waiting 0010010101111621E22F1FD 000000000,000001036s 000.00% 000.00% tasklet#0002
0004 00000000 Waiting 001001010111144144RR 000000000,007381612s 000.00% 000.00% kworker#0004
0005 00000000 Waiting 001001010111148148RR 000000000,000124382s 000.00% 000.00% kworker#H0005
0006 00000000 Running 0010010101111621E22F1FD 000000000,000523225s 000.00% 000.00% shell#0006
0008 00000001 Running 001001010111210010RR 000000268,133903237s 099.91% 100.00% idle#0008
0009 00000001 Waiting 00100101011121621E22F1FD 000000000,00000589s 000.00% 000.00% tasklet#0009
0011 00000001 Waiting 001001010111214414RR 000000000,000206619s 000.00% 000.00% kworker#0011
0012 00000001 Waiting 001001010111214814RR 000000000,000230701s 000.00% 000.00% kworker#H0012
0014 00000002 Running 001001010111410010RR 000000269,224153136s 099.92% 100.00% idle#0014
0015 00000002 Waiting 00100101011141621E22F1FD 000000000,00000589s 000.00% 000.00% tasklet#0015
0017 00000002 Waiting 001001010111414414RR 000000000,000207858s 000.00% 000.00% kworker#0017
0018 00000002 Waiting 001001010111414814RR 000000000,000119860s 000.00% 000.00% kworker#H0018
0020 00000003 Running 001001010111810010RR 000000269,134763774s 099.91% 099.99% idle#0020
0021 00000003 Waiting 001001010111621E22F1FD 000000000,00000589s 000.00% 000.00% tasklet#0021
0023 00000003 Waiting 001001010111814414RR 000000000,000206010s 000.00% 000.00% kworker#0023
0024 00000003 Waiting 001001010111814814RR 000000000,000120309s 000.00% 000.00% kworker#H0024
Current: 17/8192 I 0000 Suspend I 0000 Exiting I 0012 Waiting I 0000 Sleep I 0001 Ready I 0004 Running
CpuIdle: CPU0=99.99% CPU1=100.00% CPU2=100.00% CPU3=99.99%
ReadyCnt: CPU0=[1/5] CPU1=[0/4] CPU2=[0/4] CPU3=[0/4]

```

AArch64 (CA53 Series)

- Config/Make: make clean && make mbedtee_vexpress_ca53_defconfig && make

Support targets:

```
FVP_Base_Cortex_A53x4, FVP_Base_Cortex_A35x4, FVP_Base_Cortex_A57x4, FVP_Base_Cortex_A72x4, FVP_Base_Cortex_A73x4
```

- System Canvas - Load project: Click the "File" -> "Load Project" -> Select the **ARM/FastModelsPortfolio_11.23/examples/LISA/FVP_Base/Build_Cortex-A53x4/FVP_Base_Cortex-A53x4.sgproj**
- System Canvas - Select Active Project Configuration -> **Linux-Release-GCC-10.3** and then click the **Build** button
- System Canvas - Click the **Run** button -> Select the **ISIM system** -> Input below command to run the target -> Click **OK**

```
"%ISIM%" --data /home/kapa/mbedtee-build/buildroot/output/images/mbedtee.bin@0x80000000 --data /home/kapa/mbedtee-build/buildroot/output/images/linux.dtb@0x85f00000 --data /home/kapa/mbedtee-build/buildroot/output/images/Image@0x86000000 --start FVP_Base_Cortex_A53x4.cluster0.cpu0=0x80000000 --start FVP_Base_Cortex_A53x4.cluster0.cpu1=0x80000000 --start FVP_Base_Cortex_A53x4.cluster0.cpu2=0x80000000 --start FVP_Base_Cortex_A53x4.cluster0.cpu3=0x80000000 --param bp.secure_memory=false -S --run -C pctl.startup=0.0.* -C bp.vis.rate_limit-enable=false
```

AArch64 (big.LITTLE)

- Config/Make: make clean && make mbedtee_vexpress_ca73x4_ca53x4_defconfig && make

Support targets:

```
FVP_Base_Cortex_A73x4_A53x4, FVP_Base_Cortex_A72x4_A53x4, FVP_Base_Cortex_A57x4_A53x4  
FVP_Base_Cortex_A72x4_A35x4, FVP_Base_Cortex_A57x4_A35x4
```

- System Canvas - Load project: Click the "File" -> "Load Project" -> Select the **ARM/FastModelsPortfolio_11.23/examples/LISA/FVP_Base/Build_Cortex-A73x4-A53x4/FVP_Base_Cortex-A73x4-A53x4.sgproj**
- System Canvas - Select Active Project Configuration -> **Linux-Release-GCC-10.3** and then click the **Build** button
- System Canvas - Click the **Run** button -> Select the **ISIM system** -> Input below command to run the target -> Click **OK**

```
"%ISIM%" --data /home/kapa/mbedtee-build/buildroot/output/images/mbedtee.bin@0x80000000 --data /home/kapa/mbedtee-build/buildroot/output/images/linux.dtb@0x85f00000 --data /home/kapa/mbedtee-build/buildroot/output/images/Image@0x86000000 --start FVP_Base_Cortex_A73x4_A53x4.cluster0.cpu0=0x80000000 --start FVP_Base_Cortex_A73x4_A53x4.cluster0.cpu1=0x80000000 --start FVP_Base_Cortex_A73x4_A53x4.cluster0.cpu2=0x80000000 --start FVP_Base_Cortex_A73x4_A53x4.cluster0.cpu3=0x80000000 --start FVP_Base_Cortex_A73x4_A53x4.cluster1.cpu0=0x80000000 --start FVP_Base_Cortex_A73x4_A53x4.cluster1.cpu1=0x80000000 --start FVP_Base_Cortex_A73x4_A53x4.cluster1.cpu2=0x80000000 --start FVP_Base_Cortex_A73x4_A53x4.cluster1.cpu3=0x80000000 --param bp.secure_memory=false -S --run -C pctl.startup=0.0.*.* -C bp.vis.rate_limit-enable=false
```

AArch64 (big.LITTLE with DynamIQ)

- Config/Make: make clean && make mbedtee_vexpress_ca510x4_ca710x4_defconfig && make

Support targets:

```
FVP_Base_Cortex_A510x4_A710x4, FVP_Base_Cortex_A55x4_A75x4, FVP_Base_Cortex_A55x4_A78x4
```

- System Canvas - Load project: Click the "File" -> "Load Project" -> Select the **ARM/FastModelsPortfolio_11.23/examples/LISA/FVP_Base/Build_Cortex-A510x4+Cortex-A710x4/FVP_Base_Cortex-A510x4+Cortex-A710x4.sgproj**
- System Canvas - Select Active Project Configuration -> **Linux-Release-GCC-10.3** and then click the **Build** button
- System Canvas - Click the **Run** button -> Select the **ISIM system** -> Input below command to run the target -> Click **OK**

```
"%ISIM%" --data /home/kapa/mbedtee-build/buildroot/output/images/mbedtee.bin@0x80000000 --data /home/kapa/mbedtee-build/buildroot/output/images/linux.dtb@0x85f00000 --data /home/kapa/mbedtee-build/buildroot/output/images/Image@0x86000000 -C pctl.startup=0.0.*.* --start FVP_Base_Cortex_A510x4_Cortex_A710x4.cluster0.subcluster0.cpu0=0x80000000 --start FVP_Base_Cortex_A510x4_Cortex_A710x4.cluster0.subcluster0.cpu1=0x80000000 --start FVP_Base_Cortex_A510x4_Cortex_A710x4.cluster0.subcluster0.cpu2=0x80000000 --start FVP_Base_Cortex_A510x4_Cortex_A710x4.cluster0.subcluster0.cpu3=0x80000000 --start FVP_Base_Cortex_A510x4_Cortex_A710x4.cluster0.subcluster1.cpu0=0x80000000 --start FVP_Base_Cortex_A510x4_Cortex_A710x4.cluster0.subcluster1.cpu1=0x80000000 --start FVP_Base_Cortex_A510x4_Cortex_A710x4.cluster0.subcluster1.cpu2=0x80000000 --start FVP_Base_Cortex_A510x4_Cortex_A710x4.cluster0.subcluster1.cpu3=0x80000000 --param bp.secure_memory=false -S --run -C bp.vis.rate_limit-enable=false
```

AArch64 (SMT)

- Config/Make: make clean && make mbedtee_vexpress_ca65_defconfig && make
- Support targets: (**Simultaneous Multithreading**: Each core has 2 threads, configured to 8 cores, total 8x2=16 threads)

```
FVP_Base_Cortex_A65, FVP_Base_Cortex_A65AE, FVP_Base_Neoverse_E1
```

- System Canvas - Load project: Click the "**File**" -> "**Load Project**" -> Select the **ARM/FastModelsPortfolio_11.23/examples/LISA/FVP_Base/Build_Cortex-A65/FVP_Base_Cortex-A65.sgproj**
- System Canvas - Select Active Project Configuration -> **Linux-Release-GCC-10.3** and then click the **Build** button
- System Canvas - Click the **Run** button -> Select the **ISIM system** -> Input below command to run the target -> Click **OK**

```
"%ISIM%" --data /home/kapa/mbedtee-build/buildroot/output/images/mbedtee.bin@0x80200000 --data /home/kapa/mbedtee-build/buildroot/output/images/linux.dtb@0x85f00000 --data /home/kapa/mbedtee-build/buildroot/output/images/Image@0x86000000 --start FVP_Base_Cortex_A65.cluster0.cpu0.thread0=0x80200000 --start FVP_Base_Cortex_A65.cluster0.cpu0.thread1=0x80200000 --start FVP_Base_Cortex_A65.cluster0.cpu1.thread0=0x80200000 --start FVP_Base_Cortex_A65.cluster0.cpu1.thread1=0x80200000 --start FVP_Base_Cortex_A65.cluster0.cpu2.thread0=0x80200000 --start FVP_Base_Cortex_A65.cluster0.cpu2.thread1=0x80200000 --start FVP_Base_Cortex_A65.cluster0.cpu3.thread0=0x80200000 --start FVP_Base_Cortex_A65.cluster0.cpu3.thread1=0x80200000 --start FVP_Base_Cortex_A65.cluster0.cpu4.thread0=0x80200000 --start FVP_Base_Cortex_A65.cluster0.cpu4.thread1=0x80200000 --start FVP_Base_Cortex_A65.cluster0.cpu5.thread0=0x80200000 --start FVP_Base_Cortex_A65.cluster0.cpu5.thread1=0x80200000 --start FVP_Base_Cortex_A65.cluster0.cpu6.thread0=0x80200000 --start FVP_Base_Cortex_A65.cluster0.cpu6.thread1=0x80200000 --start FVP_Base_Cortex_A65.cluster0.cpu7.thread0=0x80200000 --start FVP_Base_Cortex_A65.cluster0.cpu7.thread1=0x80200000 --param bp.secure_memory=false -S --run -C pctl.startup=0.0.*.* -C bp.vis.rate_limit-enable=false
```

AArch32 (CA15 Series)

- Config/Make: make clean && make mbedtee_vexpress_ca15_defconfig && make
- Support targets:

```
FVP_VE_Cortex_A15x4, FVP_VE_Cortex_A7x4, FVP_VE_Cortex_A17x4
```

- System Canvas - Load project: Click the "**File**" -> "**Load Project**" -> Select the **ARM/FastModelsPortfolio_11.23/examples/LISA/FVP_VE/Build_Cortex-A15x4/FVP_VE_Cortex-A15x4.sgproj**
- System Canvas - Select Active Project Configuration -> **Linux-Release-GCC-10.3** and then click the **Build** button
- System Canvas - Click the **Run** button -> Select the **ISIM system** -> Input below command to run the target -> Click **OK**

```
"%ISIM%" --data /home/kapa/mbedtee-build/buildroot/output/images/mbedtee.bin@0x80100000 --data /home/kapa/mbedtee-build/buildroot/output/images/linux.dtb@0x85f00000 --data /home/kapa/mbedtee-build/buildroot/output/images/Image@0x86008000 --start FVP_VE_Cortex_A15x4.cluster.cpu0=0x80100000 --start FVP_VE_Cortex_A15x4.cluster.cpu1=0x80100000 --start FVP_VE_Cortex_A15x4.cluster.cpu2=0x80100000 --start FVP_VE_Cortex_A15x4.cluster.cpu3=0x80100000 --param motherboard.vis.rate_limit-enable=false
```

AArch32 (CA9 Series)

- Config/Make: make clean && make mbedtee_vexpress_ca5_defconfig && make
- Support targets:

```
FVP_VE_Cortex_A5x4, FVP_VE_Cortex_A9x4
```


- System Canvas - Load project: Click the "**File**" -> "**Load Project**" -> Select the **ARM/FastModelsPortfolio_11.23/examples/LISA/FVP_VE/Build_Cortex-A9x4/FVP_VE_Cortex-A9x4.sgproj**
- System Canvas - Select Active Project Configuration -> **Linux-Release-GCC-10.3** and then click the **Build** button
- System Canvas - Click the **Run** button -> Select the **ISIM system** -> Input below command to run the target -> Click **OK**

```
"%ISIM%" --data /home/kapa/mbedtee-build/buildroot/output/images/mbedtee.bin@0x80000000 --data /home/kapa/mbedtee-build/buildroot/output/images/linux.dtb@0x85f00000 --data /home/kapa/mbedtee-build/buildroot/output/images/Image@0x86008000 --start FVP_VE_Cortex_A9x4.cluster.cpu0=0x80000000 --start FVP_VE_Cortex_A9x4.cluster.cpu1=0x80000000 --start FVP_VE_Cortex_A9x4.cluster.cpu2=0x80000000 --start FVP_VE_Cortex_A9x4.cluster.cpu3=0x80000000 --param motherboard.vis.rate_limit-enable=false
```

AArch32 (big.LITTLE)

- Config/Make: make clean && make mbedtee_vexpress_ca17x4_ca7x4_defconfig && make
- Support targets:

```
FVP_VE_Cortex_A17x4_A7x4, FVP_VE_Cortex_A15x4_A7x4
```

- System Canvas - Load project: Click the "**File**" -> "**Load Project**" -> Select the **ARM/FastModelsPortfolio_11.23/examples/LISA/FVP_VE/Build_Cortex-A17x4-A7x4/FVP_VE_Cortex-A17x4-A7x4.sgproj**
- System Canvas - Select Active Project Configuration -> **Linux-Release-GCC-10.3** and then click the **Build** button
- System Canvas - Click the **Run** button -> Select the **ISIM system** -> Input below command to run the target -> Click **OK**

```
"%ISIM%" --data /home/kapa/mbedtee-build/buildroot/output/images/mbedtee.bin@0x80000000 --data /home/kapa/mbedtee-build/buildroot/output/images/linux.dtb@0x85f00000 --data /home/kapa/mbedtee-build/buildroot/output/images/Image@0x86008000 -C FVP_VE_Cortex_A17x4_A7x4.coretile.dualclustersystemconfigurationblock.CFG_ACTIVECLUSTER=3 --param motherboard.vis.rate_limit-enable=false --start FVP_VE_Cortex_A17x4_A7x4.coretile.cluster0.cpu0=0x80000000 --start FVP_VE_Cortex_A17x4_A7x4.coretile.cluster0.cpu1=0x80000000 --start FVP_VE_Cortex_A17x4_A7x4.coretile.cluster0.cpu2=0x80000000 --start FVP_VE_Cortex_A17x4_A7x4.coretile.cluster0.cpu3=0x80000000 --start FVP_VE_Cortex_A17x4_A7x4.coretile.cluster1.cpu0=0x80000000 --start FVP_VE_Cortex_A17x4_A7x4.coretile.cluster1.cpu1=0x80000000 --start FVP_VE_Cortex_A17x4_A7x4.coretile.cluster1.cpu2=0x80000000 --start FVP_VE_Cortex_A17x4_A7x4.coretile.cluster1.cpu3=0x80000000 -S --run
```

Post-Build Validation

After launching MbedTEE on any platform, the following tests **must** all pass before the build can be considered correct. Never declare a build successful based solely on the presence of the boot log -- always close the loop with the tests below.

Step 1 -- MbedTEE Crypto Self-Tests (all platforms)

Run from the **MbedTEE console** (telnet port 5556 / 6556 / 7556, or stdio for TEE-only platforms):

```
mbedtest -c
```

Pass criterion: Output ends with `Results: N passed, 0 failed`. Any failure indicates a crypto library or build issue and must be resolved before proceeding.

Step 2 -- MbedTEE Full Kernel Tests (all platforms)

Run from the **MbedTEE console**:

```
mbedtest --test &
```

Pass criterion: No `oops` message and no `TEST FAIL` line anywhere in the output. Wait for the command to finish before declaring pass. For stress testing, multiple instances can be launched concurrently:

```
mbedtest --test &  
mbedtest --test &
```

Step 3 -- GlobalPlatform Compliance Test (REE+TEE platforms only)

Applicable to platforms that run Linux (REE) alongside MbedTEE -- e.g. AArch32, AArch64, and RISC-V REE+TEE configurations. Run from the **LinuxREE console** (telnet port 5555 / 6555, or the REE serial):

```
mbedtee-gp-client -r 1
```

`-r 1` runs one complete round of all 2350 GlobalPlatform test cases. **Pass criterion:** All cases complete with no failure message and no crash.

Note

Steps 1 and 2 are **mandatory for every platform** without exception. Step 3 is mandatory for any platform that includes a Linux REE component. A platform is only considered validated when all applicable steps pass.

GlobalPlatform

MbedTEE GlobalPlatform TestSuite is ported from **TEE_Initial_Configuration-Test_Suite_v2_0_0_2-2017_06_09.7z**, and it is placed in two git repositories, one is for REE client, another one is for TEE TTAs:

```
https://github.com/mbedtee/mbedtee-globalplatform-client.git
https://github.com/mbedtee/mbedtee-globalplatform-ta.git
```

If you are a member of GlobalPlatform or you have purchased the TEE_Initial_Configuration-Test_Suite, you can email the author to request access to these two git repositories.

The GlobalPlatform TestSuite can be enabled through buildroot menuconfig, buildroot clones them from the above repositories and then builds them:

make menuconfig -> Enter the "TEE" menu -> Select the "**mbedtee GlobalPlatform client**" and "**mbedtee GlobalPlatform TA**" -> **make all**

```

                                TEE
Arrow keys navigate the menu. <Enter> selects submenus ---> (or empty submenus ----).
Highlighted letters are hotkeys. Pressing <Y> selects a feature, while <N> excludes a
feature. Press <Esc><Esc> to exit, <?> for Help, </> for Search. Legend: [*] feature is
selected [ ] feature is excluded
↑(-)
[*] mbedtee supplicant
-* mbedtee cryptotool
[*] mbedtee client API
[*] mbedtee client APP
[*] mbedtee HelloWorld client
[*] mbedtee GlobalPlatform client
[*] mbedtee TA
[*] mbedtee HelloWorld TA
[*] mbedtee GlobalPlatform TA
(vexpress_ca17x4_ca7x4) mbedtee platform
↓(+)

<Select>  < Exit >  < Help >  < Save >  < Load >
```

After successfully building the GlobalPlatform TestSuite, you can run the test via **mbedtee-gp-client** application at LinuxREE terminal:

```
# mbedtee-gp-client -h
VERSION: 1.0
mbedtee-gp-client [-h|-s case|-r n] -e

h: help.
e: errexit on any test case failure
r: repeat times of test rounds, default -1.
s: specified the test case
   if no specified case, all[2350] cases will be tested

d3-26-21 d3-fe-f9 d3-de-ad d3-28-c9 d3-87-46 29-ff-45 29-85-65 29-a6-7a
29-7a-e2 29-8e-75 29-ab-17 29-13-74 29-4c-5c 29-0c-d9 29-d1-8f 29-60-85
.....
```

```
LinuxREE MbedTEE
b6-54-fd passed
b6-1d-00 Round.1
b6-1d-00 passed
b6-1d-01 Round.1
b6-1d-01 passed
b6-10-01 Round.1
b6-10-01 passed
b6-10-00 Round.1
b6-10-00 passed
b6-1e-ec Round.1
b6-1e-ec passed
b6-1e-ed Round.1
b6-1e-ed passed
b6-1e-eb Round.1
b6-1e-eb passed
b6-1e-ea Round.1
b6-1e-ea passed
Round.1 2218 out of 2218 passed
# mbedtee-gp-client -r 1 -e
[ERR 2807|2807@CPU02]TEE_CopyObjectAttributes1 (0942): TEE_Panic - 0x16 (22)
[ERR 1010|0000@CPU01]rpc_session_close (0441): No such session 2807
[ERR 2808|2808@CPU03]TEE_SetOperationKey (0664): TEE_Panic - 0x16 (22)
[ERR 2342|0000@CPU01]rpc_session_close (0441): No such session 2808
[ERR 2809|2809@CPU00]TEE_SetOperationKey (0664): TEE_Panic - 0x16 (22)
[ERR 2342|0000@CPU01]rpc_session_close (0441): No such session 2809
[ERR 2810|2810@CPU02]TEE_SetOperationKey (0664): TEE_Panic - 0x16 (22)
[ERR 1010|0000@CPU01]rpc_session_close (0441): No such session 2810
[ERR 2811|2811@CPU03]TEE_SetOperationKey (0664): TEE_Panic - 0x16 (22)
[ERR 1010|0000@CPU01]rpc_session_close (0441): No such session 2811
[ERR 2812|2812@CPU00]TEE_SetOperationKey (0664): TEE_Panic - 0x16 (22)
[ERR 2342|0000@CPU01]rpc_session_close (0441): No such session 2812
[ERR 2813|2813@CPU02]TEE_SetOperationKey (0664): TEE_Panic - 0x16 (22)
[ERR 1010|0000@CPU01]rpc_session_close (0441): No such session 2813
[ERR 2815|2815@CPU00]TEE_SetOperationKey (0664): TEE_Panic - 0x16 (22)
[ERR 2342|0000@CPU01]rpc_session_close (0441): No such session 2815
[ERR 2817|2817@CPU03]TEE_SetOperationKey (0664): TEE_Panic - 0x16 (22)
[ERR 2347|0000@CPU01]rpc_session_close (0441): No such session 2817
[ERR 2819|2819@CPU02]TEE_SetOperationKey (0664): TEE_Panic - 0x16 (22)
[ERR 2342|0000@CPU01]rpc_session_close (0441): No such session 2819
```

Adding TA

In general, you can add your own trusted applications by referencing the **mbedtee-helloworld-ta** and **mbedtee-helloworld-client**.

- Top level Makefile @ buildroot - mbedtee-helloworld-ta.mk:
 - specifies the cross-toolchain and architecture-specific compile options
 - encrypts and signs the compiled TA ELF object

```
kapa@ubuntu24:~/mbedtee-build/buildroot$ tree tee/mbedtee/mbedtee-ta/mbedtee-helloworld-ta/
tee/mbedtee/mbedtee-ta/mbedtee-helloworld-ta/
|-- Config.in
`-- mbedtee-helloworld-ta.mk
```

- Lower level source code @ TA source tree:
 - provide the TA source and configuration
 - compile the TA ELF object
 - Protocol V2: the TA ELF is encrypted and signed using the **mbedtee-crypto** tool with selectable algorithms:
 - **Signature algorithms** (RSA-PSS, ECDSA, Ed25519):
 - `rsa2048-pss-sha256`, `rsa3072-pss-sha256`, `rsa3072-pss-sha384`, `rsa4096-pss-sha256`, `rsa4096-pss-sha384`, `rsa4096-pss-sha512`
 - `ecdsa-p256-sha256`, `ecdsa-p384-sha384`, `ecdsa-p521-sha512`
 - `ecdsa-bp256-sha256`, `ecdsa-bp384-sha384`, `ecdsa-bp512-sha512` (Brainpool curves)
 - `ed25519`
 - **Encryption algorithms** (AEAD):
 - `aes-256-gcm`, `sm4-gcm`, `chacha20-poly1305`
 - The signing/encryption flow (`auth_ta` uses **verify-then-decrypt**):
 1. Verify the TA object's signature using the public key embedded in the certificate
 2. AEAD decrypt the TA body (authenticated decryption with integrity check via GCM/Poly1305 tag)
 3. Install the decrypted TA ELF and set its configuration
 - Two output files are generated by `mbedtee-crypto`:
 - `.certi` - certificate containing public key, encrypted symmetric key, TA config, and its own signature
 - `.o` - TA object: `ta_obj_header` + AEAD-encrypted ELF body + signature over (header || encrypted body)

```
kapa@ubuntu24:~/mbedtee-build/buildroot$ tree dl/mbedtee-helloworld-ta/git
dl/mbedtee-helloworld-ta/git
|-- LICENSE
|-- Makefile
|-- mbedtee-helloworld-ta.c
|-- mbedtee-helloworld-ta.config
`-- README.md

`-- signed
   |-- mbedtee-helloworld-ta.certi
   `-- mbedtee-helloworld-ta.o
```

- TA configuration @ mbedtee-helloworld-ta.config:
 - provide the basic TA information, e.g. Name/UUID/ELF-Path etc.

```
name = "mbedtee-helloworld-ta";
uuid = "6d809454-ccb2-4ed9-b7cf-92447354b367";
path = "/user/mbedtee-helloworld-ta.o";
version = "1.0.0";
stack_size = "8192";
heap_size = "1048576";
single_instance = "1";
multi_session = "1";
dev_access = "/dev/uart0, /dev/uart1, /dev/urandom, /dev/globalplatform";
description = "Hello world";
```

Important

The MbedTEE ELF loader needs the **.symtab** and **.strtab** sections to locate the GlobalPlatform specified TA entries, such as

(*TA_CreateEntryPoint/TA_DestroyEntryPoint/TA_OpenSessionEntryPoint/TA_CloseSessionEntryPoint/TA_InvokeCommandEntryPoint*), so please DO NOT strip these two sections in the compiled TA ELF object, usually you can refer to the method in `Makefile@mbedtee-helloworld-ta`:

`$(STRIP) -d -R .comment ${APP}` ## this strip option removes the unnecessary sections and keeps the **`.symtab`** and **`.strtab`** sections.

Debugging

Backtrace

- Enable the user space and kernel space backtrace through `make mbedtee-os-menuconfig`
- Example of a fault from user space:

```
[INF 0081|0058@CPU02]backtrace
#1      <00000049f8f23f38>      (0138):
#2      <00000049f8d40890>      (shm_open + 0xbc)
#3      <00000049f8d4250c>      (shmfs_mmap1.isra.0 + 0x84)
#4      <00000049f8d42a9c>      (fs_test_entry + 0x68)
#5      <00000049f8f24f2c>      (t2_routine + 0xc4)
                                (pthread_entry + 0x2c)
```

- Example of a fault from kernel space:

```
auth_ta@0053|0027 backtrace - kernel
#0      <ffffffffffc00024520>      (fat_rename + 0x150)
#1      <ffffffffffc0002174>      (sys_rename + 0x7c)
#2      <ffffffffffc00006964>      (do_syscall_rename + 0x50)
#3      <ffffffffffc0004b764>      (syscall_handler + 0x94)
#4      <ffffffffffc0003441c>      (synchronous_el0 + 0x1c)
[INF 0053|0027@CPU02]backtrace - user
#1      <00000044150b18dc>      (0138):
#2      <00000044150afb5c>      (__syscall + 0x8)
#3      <0000004414ecbae8>      (_rename_r + 0x1c)
#4      <0000004414ecf508>      (fstest + 0x6c8)
#5      <0000004414ecfa9c>      (fs_test_entry + 0x64)
#6      <00000044150b1f24>      (t2_routine + 0xc4)
                                (pthread_entry + 0x2c)
```

Oops

Kernel exception handler is able to provide the exception information and dump the registers:

```
[ERR-0053|0027@CPU02]__oops      (0169): address: 0xffffffffffffffff
[ERR-0053|0027@CPU02]__oops      (0170): sp 0xffffffffbfc024f930
[ERR-0053|0027@CPU02]__oops      (0171): spsr 0x60000304
[ERR-0053|0027@CPU02]__oops      (0172): exception class: 0x25
[ERR-0053|0027@CPU02]__oops      (0173): exception iss: 0x45
[ERR-0053|0027@CPU02]__oops      (0174): encoding: DABT (current EL)
[ERR-0053|0027@CPU02]__oops      (0178): oops@auth_ta@0053|0027 asid 3 usp(0x2000@0x4414e8f000)
[ERR-0053|0027@CPU02]__oops      (0187): pc 0xffffffffc00024520 (fat_rename + 150)
[ERR-0053|0027@CPU02]__oops      (0190): lr 0xffffffffc0002451c (fat_rename + 14c)

x[00~03] ffffffff 0000000000000000 0000000000000001 0000000000000000
x[04~07] ffffffffbfc024fbc8 0000000000000004 4f481f40533d2c3b 7f7f7f7f7f7f7f7f
x[08~11] 0101010101010101 000000000000002f 000000000000002f 0000000000000000
x[12~15] 00030000401b7000 0000004414e90a88 0000000000000300 ffffffffbfc024e000
x[16~19] 0000004415164620 00000044150b18d4 0000000000000000 00000000ffffff0
x[20~23] ffffffffbfc0094a08 ffffffffc000276215 ffffffffbfc0094970 ffffffffc0000243d0
x[24~27] ffffffffc0000523ba 0000004414e90bc8 0000004414e90dc8 00000000000001092
x[28~29] 000000000000000a ffffffffbfc024fbd0
```

Shell

MbedTEE provides a simple shell which can provide some basic commands to query the system information:

- `ps/ls/mem/rm/kill/irq/date/mount` etc.

```
# ps
TID|PID@CPU      STATE  SG|UC|KC|E|B|A|PRI@POLICY      RUNTIME      LOADING      NAME
0001|0000@0      Ready  00|00|01|0|1|1|00|00@RR      000000309.215397304s  099.89% 099.96% idle@0001
0002|0000@0      Waiting 00|00|01|0|1|1|62|62@FIFO      000000000.000623288s  000.00% 000.00% taskletd@0002
0004|0000@0      Waiting 00|00|01|0|1|1|44|44@RR      000000000.096608240s  000.03% 000.00% kworker@0004
0005|0000@0      Waiting 00|00|01|0|1|1|48|48@RR      000000000.013861808s  000.00% 000.00% kworker-H@0005
0006|0000@0      Running 00|00|01|0|0|f|62|62@FIFO      000000000.010114440s  000.00% 000.03% shell@0006|0006
0008|0000@1      Running 00|00|01|0|1|2|00|00@RR      000000309.127846776s  099.87% 099.98% idle@0008
0009|0000@1      Waiting 00|00|01|0|1|2|62|62@FIFO      000000000.000018064s  000.00% 000.00% taskletd@0009
0011|0000@1      Waiting 00|00|01|0|1|2|44|44@RR      000000000.007915272s  000.00% 000.00% kworker@0011
0012|0000@1      Waiting 00|00|01|0|1|2|48|48@RR      000000000.013388112s  000.00% 000.00% kworker-H@0012
0014|0000@2      Running 00|00|01|0|1|4|00|00@RR      000000308.507188944s  099.67% 099.99% idle@0014
0015|0000@2      Waiting 00|00|01|0|1|4|62|62@FIFO      000000000.000010952s  000.00% 000.00% taskletd@0015
0017|0000@2      Waiting 00|00|01|0|1|4|44|44@RR      000000000.007978960s  000.00% 000.00% kworker@0017
0018|0000@2      Waiting 00|00|01|0|1|4|48|48@RR      000000000.012593160s  000.00% 000.00% kworker-H@0018
0020|0000@3      Running 00|00|01|0|1|8|00|00@RR      000000308.825292960s  099.77% 099.99% idle@0020
0021|0000@3      Waiting 00|00|01|0|1|8|62|62@FIFO      000000000.000011344s  000.00% 000.00% taskletd@0021
0023|0000@3      Waiting 00|00|01|0|1|8|44|44@RR      000000000.008045872s  000.00% 000.00% kworker@0023
0024|0000@3      Waiting 00|00|01|0|1|8|48|48@RR      000000000.024208920s  000.00% 000.00% kworker-H@0024
Current: 17/8192 | 0000 Suspend | 0000 Exiting | 0012 Waiting | 0000 Sleep | 0001 Ready | 0004 Running
Cpuidle: CPU0=99.96% CPU1=99.98% CPU2=99.99% CPU3=99.99%
ReadyCnt: CPU0=[1/5] CPU1=[0/4] CPU2=[0/4] CPU3=[0/4]
```

```
# mem
Buddy Pools: 000009, Free 0x00001480 SingleAllocMax: 0x400
Bitmap Pools: 000008, Free 0x00003c60 SingleAllocMax: 0xfc0

page pool 0 info: size 0x10000000
page phys: 0x40000000, va: 0xffffffffbfc0000000
page cnt: 65050/65536 (singleAllocMax: 32768)

page pool 1 info: size 0x5572000
page phys: 0x8028e000, va: 0xffffffffc00028e000
page cnt: 21836/21874 (singleAllocMax: 16384)

kvma info: size 2000000000, va: 0xfffffffff9f800000000
1 0 Start 0xfffffffff9f800000000 Size 80000, Free 3d000, MaxOrder 17
1 1 Start 0xfffffffff9f800000000 Size 4000000, Free 3c80000, MaxOrder 25
1 2 Start 0xfffffffff9f800000000 Size 200000000, Free 1fc000000, MaxOrder 32
1 3 Start 0xfffffffff9f800000000 Size 2000000000, Free 1e00000000, MaxOrder 36
```

```
# irq
irq      hwirq      parent  childs  enabled affinity  total-cnt  percpu-cnt      controller
1        9          root    1        1         f          1513316      CPU00: 476313    riscv,ac1int
          CPU01: 340541
          CPU02: 351905
          CPU03: 344557

2        5          root    0         1         f          321849      CPU00: 77923     riscv,ac1int
          CPU01: 82682
          CPU02: 78224
          CPU03: 83020

3        1          root    0         1         f          1696388     CPU00: 494845    riscv,imsic
          CPU01: 401427
          CPU02: 404184
          CPU03: 395932

6        12         7        0         1         3           0          CPU00: 0         riscv,aplic
          CPU01: 0
          CPU02: 0
          CPU03: 0

7        29         1        1         1         3          204         CPU00: 128       riscv,imsic
          CPU01: 60
          CPU02: 12
          CPU03: 4

IPI:    1759276      RPC_CALLEE: 279686      RPC_CALLER: 356419
CPU00: 247037      CPU00:    269297      CPU00:    285497
CPU01: 502732      CPU01:    3452        CPU01:    24715
CPU02: 506899      CPU02:    3721        CPU02:    23772
CPU03: 502608      CPU03:    3216        CPU03:    22435
```

Misc

- ASLR is enabled by default; it is better to disable it through [menuconfig](#) if you are debugging issues related to memory faults.
- The default trace level of kernel is `TRACE_LEVEL_INFO`, you can change the `CONFIG_TRACE_LEVEL` through [menuconfig](#).
- The default trace level of user is `TRACE_LEVEL_ERROR`, you can change it in **utrace.h**.
- The default layout of address spaces is defined in the architecture-specific **map.h** (e.g. `arch/arm/aarch64/include/map.h`), you can check and change them if needed.